

YuriiFormer

A Suite of Nesterov-Accelerated Transformers

Zimin, Polyanskiy & Rigollet *arXiv:2601.23236, Mar 2026*

Presented by: Minjie Cheng

May 27, 2026

Outline

- 1 Motivation
- 2 Transformers as Optimizers
- 3 Nesterov Acceleration
- 4 Experiments
- 5 Extensions
- 6 Discussion

Why Revisit Transformer Architecture Design?

The Problem

- Transformers (Attn + MLP + residual + LN) are empirically powerful, but their architecture is largely **empirically designed** without principled theoretical guidance.

This Paper's Approach

- View transformer layers as **iterations of an optimization algorithm** on token embeddings
- This unlocks a principled design space via classical optimization theory

Key Insight

Standard GPT $\equiv$ **Vanilla Gradient Descent** on a composite energy, via **Lie–Trotter splitting**.

Replace GD with **Nesterov acceleration** $\Rightarrow$ **YuriiFormer**

Transformers as Optimization Algorithms

Unified Variational Framework

Each transformer layer = one discrete step of a **first-order optimizer** on an implicit objective over all token embeddings.

Token configuration:

$$X := (x_1, \dots, x_n) \in (\mathbb{R}^d)^n$$

- **Attention** layer $\leftrightarrow$ gradient oracle for **interaction energy** $\mathcal{E}(X)$ (token-token)
- **MLP** layer (Feed Forward Network) $\leftrightarrow$ gradient oracle for **potential energy** $\mathcal{F}(X)$ (token-wise)
- **Depth** $\leftrightarrow$ iteration count

Composite objective:

$$\text{minimize } \mathcal{E}(X) + \mathcal{F}(X)$$

Transformer block = one step of **gradient descent** on $\mathcal{E} + \mathcal{F}$
implemented via a **splitting scheme**

Attention as Gradient of Interaction Energy

Single attention layer with residual connection:

$$x_i \leftarrow x_i + \frac{\sum_{j=1}^n V_t x_j e^{\langle Q_t x_i, K_t x_j \rangle}}{\sum_{j=1}^n e^{\langle Q_t x_i, K_t x_j \rangle}} \quad (1)$$

Interaction energy:

$$\mathcal{E}(X) := \sum_{i,j=1}^n e^{\langle x_i, x_j \rangle}$$

Gradient w.r.t. token i under a Riemannian metric:

$$\nabla_{x_i} \mathcal{E}(X) = \frac{\sum_j x_j e^{\langle x_i, x_j \rangle}}{\sum_j e^{\langle x_i, x_j \rangle}}$$

Eq. (1) = modulated gradient step:

① **Preconditioning:** $\nabla_{x_i} \mathcal{E} \mapsto V_t \nabla_{x_i} \mathcal{E}$

② **Coord. change:**

$$\langle x_i, x_j \rangle \mapsto \langle Q_t x_i, K_t x_j \rangle$$

At full-config level: $X \leftarrow X + \text{Attn}_t(X)$

MLP as Gradient of Potential Energy

MLP residual update (acts independently on each token):

$$x_i \leftarrow x_i + W_t \sigma(A_t x_i + b_t), \quad i = 1, \dots, n \quad (2)$$

Potential energy:

$$\mathcal{F}(X) = \sum_{i=1}^n V(x_i), \quad V(x) := \sum_{\ell=1}^d v(x^{(\ell)})$$

where $v : \mathbb{R} \rightarrow \mathbb{R}$ is scalar with $v' = \sigma$.

Then $\nabla V(x) = \sigma(x)$ and:

$$X \leftarrow X + \text{MLP}_t(X)$$

MLP = gradient step on $\mathcal{F}$, modulated by layer-dependent preconditioner W_t and affine reparameterization $x \mapsto A_t x + b_t$.

Complementary roles:

- Attn $\rightarrow$ *mixes across tokens*
- MLP $\rightarrow$ *acts per token*

Composite Optimization & Splitting Schemes

Joint ODE for the token configuration:

$$\dot{X}(t) = \text{Attn}_t(X(t)) + \text{MLP}_t(X(t)) \quad (3)$$

Forward Euler (parallel update):

$$X \leftarrow X + \text{Attn}_t(X) + \text{MLP}_t(X) \quad (4)$$

Used in PaLM-style models.

Lie–Trotter splitting (sequential):

$$X \leftarrow X + \text{Attn}_t(X), \quad X \leftarrow X + \text{MLP}_t(X) \quad (5)$$

Standard GPT-style architecture

Central Observation

Standard GPT blocks = **vanilla GD** on $\mathcal{E} + \mathcal{F}$ via Lie–Trotter.

Architecture design = choosing (1) an **optimization template** and (2) a **splitting scheme**.

Nesterov's Accelerated Gradient (NAG)

NAG (Nesterov, 1983) achieves convergence rate $O(1/t^2)$ vs. $O(1/t)$ for GD on smooth convex functions.

Three steps at each iteration:

$$x_{t+\frac{1}{2}} = x_t + \mu_t v_t \quad (\text{Lookahead})$$

$$v_{t+1} = \beta_t v_t - \gamma_t \nabla f(x_{t+\frac{1}{2}}) \quad (\text{Velocity update})$$

$$x_{t+1} = x_t + v_{t+1} \quad (\text{State update})$$

Key feature: gradient evaluated at **lookahead point** $x_{t+\frac{1}{2}}$, not current x_t

vs. Polyak Heavy-Ball

Polyak omits the lookahead step:

$$v_{t+1} = \beta_t v_t - \gamma_t \nabla f(x_t)$$

$$x_{t+1} = x_t + v_{t+1}$$

($\mu_t = 0$ in NAG)

In YuriiFormer:

Replace $\nabla f(\cdot)$ with $\text{Attn}_t(\cdot)$ and $\text{MLP}_t(\cdot)$, keeping the *same oracle structure*.

YuriiFormer with Euler Discretization

State: token config X_t + token velocity $V_t \in (\mathbb{R}^d)^n$

Three steps per block (NAG + Forward Euler):

$$\begin{aligned}X_t^{\text{in}} &= X_t + \mu_t V_t \quad (\text{lookahead}) \\V_{t+1} &= \beta_t V_t + \gamma_t \text{Attn}_t(X_t^{\text{in}}) + \gamma_t \text{MLP}_t(X_t^{\text{in}}) \\X_{t+1} &= X_t + V_{t+1}\end{aligned}$$

- $\mu_t, \beta_t \in (0, 1)$ are **momentum parameters**
- $\gamma_t > 0$ is a **step size**
- All learned end-to-end
- **Same** Attn/MLP oracle calls as baseline (no extra compute)
- Velocity LayerNorm applied after velocity update
- Pre-LayerNorm layout used in practice

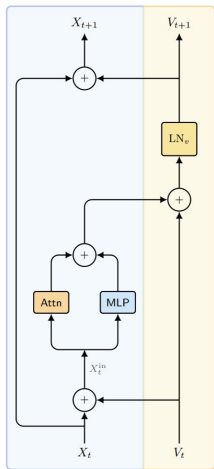
YuriiFormer with Lie–Trotter Splitting (Main Variant)

Nesterov acceleration + Lie–Trotter $\Rightarrow$ sequential Attn then MLP updates:

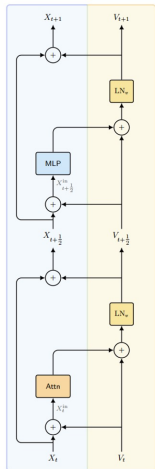
$$\begin{aligned}X_t^{\text{in}} &= X_t + \mu_t V_t \\V_{t+\frac{1}{2}} &= \beta_t V_t + \gamma_t \text{Attn}_t(X_t^{\text{in}}) \\X_{t+\frac{1}{2}}^{\text{in}} &= X_t + V_{t+\frac{1}{2}} + \mu_{t+\frac{1}{2}} V_{t+\frac{1}{2}} \\V_{t+1} &= \beta_{t+\frac{1}{2}} V_{t+\frac{1}{2}} + \gamma_{t+\frac{1}{2}} \text{MLP}_t(X_{t+\frac{1}{2}}^{\text{in}}) \\X_{t+1} &= X_{t+\frac{1}{2}} + V_{t+1}\end{aligned}$$

- Mirrors standard GPT block structure (Attn $\rightarrow$ MLP sequentially)
- Injects **momentum at the representation level** via two-stream (X, V)
- Velocity LayerNorm LN_v applied after each velocity update
- **Best performing** variant in experiments

Architecture: Two-Stream Transformer Block



Euler Discretization



Lie-Trotter Splitting

Key points:

- Two parallel streams: X (position) and V (velocity)
- V_0 initialized from separate token + positional embedding tables
- Same Attn/MLP modules as baseline
- Added params: velocity LN ($O(Ld)$) + $O(L)$ learned scalars

Experimental Setup

Baseline: nanoGPT (Karpathy, 2022), trained with **Muon** optimizer

Two datasets:

- **TinyStories** (TS): 10k steps, 4.92B tokens
- **OpenWebText** (OWT): 30k steps, 14.75B tokens

Two model sizes:

- **small:** 12L/12H/768d – 123.6M params
- **medium:** 24L/16H/1024d – 353.6M params

Six methods compared:

- ① GD + Euler
- ② GD + Lie–Trotter (**nanoGPT**)
- ③ Polyak + Euler
- ④ Polyak + Lie–Trotter
- ⑤ Nesterov + Euler
- ⑥ **Nesterov + Lie–Trotter (best)**

Identical training budgets; same batch/seq length; same data order across all methods.

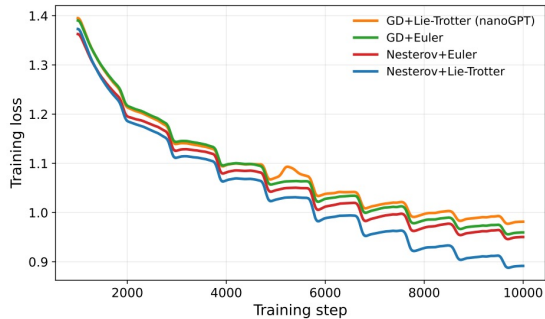
Results: TinyStories (small, 10k steps)

Method	Best	Final	Train@10k
GD + Euler	1.107	1.113	0.962
Polyak + Euler	1.099	1.110	0.933
Nesterov + Euler	1.101	1.112	0.953
GD + Lie–Trotter	1.106	1.114	0.985
Polyak + Lie–Trotter	1.083	1.096	0.899
Nesterov + Lie–Trotter	1.078	1.090	0.896

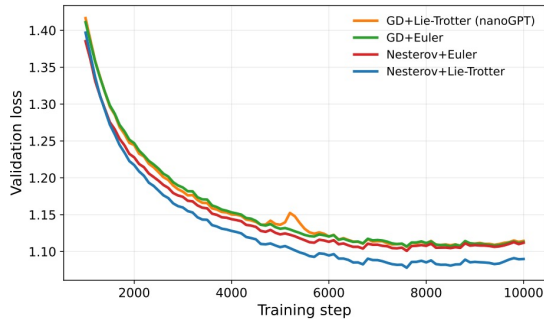
Table: Validation cross-entropy, next-token prediction losses (Best = min validation loss; Final = validation loss at 10k steps; Train @10k = training loss at 10k.)

- Nesterov + Lie–Trotter achieves **lowest** best & final val loss
- Val loss reaches minimum ~ 7.6 k steps for all; training loss continues to decrease (overfitting late)
- Nesterov lookahead provides small but **consistent gain** over Polyak

Results: TinyStories (small, 10k steps)



(a) **Training loss.**



(b) **Validation loss.**

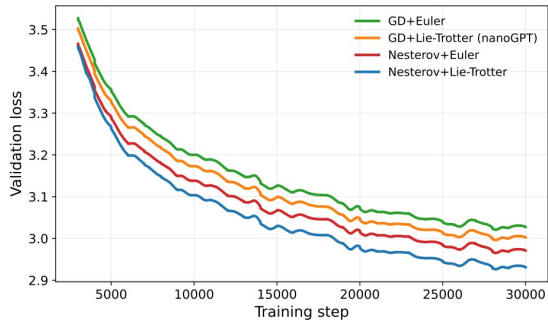
Results: OpenWebText (30k steps)

Method	small		medium	
	Best	Final	Best	Final
GD + Euler	3.014	3.023	2.770	2.775
Polyak + Euler	2.965	2.972	2.730	2.734
Nesterov + Euler	2.959	2.966	2.730	2.731
GD + LT	2.990	2.998	2.758	2.764
Polyak + LT	2.925	2.931	2.702	2.710
Nesterov + LT	2.920	2.926	2.702	2.707

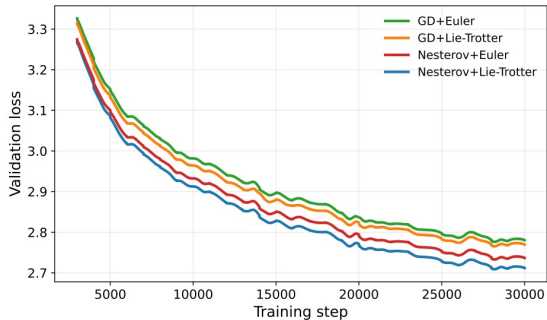
Table: Val loss (nats/token; lower is better)

- **Consistent ordering** across both sizes: GD+Euler highest, Nesterov+LT lowest
- Ordering stabilizes after 3k warmup steps
- small: Nesterov+LT reaches **2.920**, below GPT-2 (3.12) with 50× fewer tokens
- medium: **2.702**, below GPT-2 medium (2.84)
- Lie–Trotter > Euler more pronounced on OWT

Results: OpenWebText (30k steps)



(a) small



(b) medium

Downstream Task Performance (OpenWebText)

Table 4. OpenWebText: length-normalized accuracy (acc_{norm} ; %; higher is better) on HellaSwag (HS) and ARC-Easy (ARC) using the Language Model Evaluation Harness, evaluated at the best-validation checkpoint. Few-shot uses 10-shot HellaSwag and 25-shot ARC-Easy; 0-shot results are shown for reference. Errors are standard errors. Bold indicates the top-two methods in each column; ties at the reported precision are boldfaced (e.g., small ARC 0-shot has a tie for second at 41.1%).

METHOD	small				medium			
	10/25-SHOT		0-SHOT		10/25-SHOT		0-SHOT	
	HS	ARC	HS	ARC	HS	ARC	HS	ARC
GD+EULER	29.9 $\pm$ 0.5	39.8 $\pm$ 1.0	30.2 $\pm$ 0.5	38.5 $\pm$ 1.0	34.7 $\pm$ 0.5	46.0 $\pm$ 1.0	34.4 $\pm$ 0.5	40.5 $\pm$ 1.0
POLYAK+EULER	30.9 $\pm$ 0.5	41.3 $\pm$ 1.0	30.5 $\pm$ 0.5	39.8 $\pm$ 1.0	36.3 $\pm$ 0.5	48.2 $\pm$ 1.0	36.1 $\pm$ 0.5	43.4 $\pm$ 1.0
NESTEROV+EULER	31.2 $\pm$ 0.5	41.6 $\pm$ 1.0	30.5 $\pm$ 0.5	41.2$\pm$1.0	36.3 $\pm$ 0.5	48.0 $\pm$ 1.0	35.4 $\pm$ 0.5	43.5 $\pm$ 1.0
GD+LIE-TROTTER	30.0 $\pm$ 0.5	40.5 $\pm$ 1.0	29.7 $\pm$ 0.5	39.4 $\pm$ 1.0	35.5 $\pm$ 0.5	45.4 $\pm$ 1.0	35.2 $\pm$ 0.5	42.6 $\pm$ 1.0
POLYAK+LIE-TROTTER	31.6$\pm$0.5	42.5$\pm$1.0	31.2$\pm$0.5	41.1$\pm$1.0	37.1$\pm$0.5	49.0$\pm$1.0	36.8$\pm$0.5	44.0$\pm$1.0
NESTEROV+LIE-TROTTER	31.8$\pm$0.5	42.6$\pm$1.0	31.2$\pm$0.5	41.1$\pm$1.0	36.8$\pm$0.5	48.5$\pm$1.0	36.5$\pm$0.5	45.7$\pm$1.0

Extensions: Polyak

Polyak with Lie-Trotter ($\mu_t = 0$):

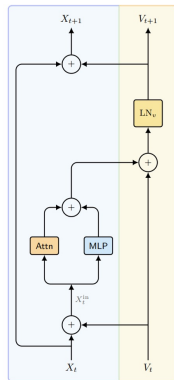
$$V_{t+\frac{1}{2}} = \beta_t V_t + \gamma_t \text{Attn}_t(X_t)$$

$$X_{t+\frac{1}{2}} = X_t + V_{t+\frac{1}{2}}$$

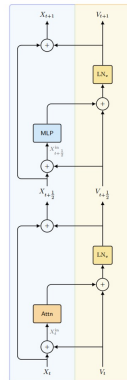
$$V_{t+1} = \beta_{t+\frac{1}{2}} V_{t+\frac{1}{2}} + \gamma_{t+\frac{1}{2}} \text{MLP}_t(X_{t+\frac{1}{2}})$$

$$X_{t+1} = X_{t+\frac{1}{2}} + V_{t+1}$$

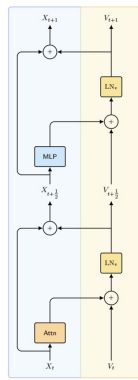
Separate learned scalars for each substep.



Nesterov+ Euler



Nesterov+Lie-Trotter



Polyak + Lie-Trotter

Limitations & Open Questions

Theoretical limitations:

- No formal convergence/optimality guarantees
- Implicit objectives from learned Attn/MLP oracles are **nonconvex** and layer-dependent
- Classical optimization theory guarantees *do not* directly transfer
- Optimization analogies serve as **design heuristics**, not literal solvers

Empirical limitations:

- Evaluation on small- and medium-scale models only ($\leq 354\text{M}$ params)
- Larger scales, longer contexts remain untested

Open questions:

- Does the benefit scale to billion-parameter models?
- Can the framework guide non-momentum innovations (e.g., second-order methods)?

Summary of Contributions

1. Variational Framework

Transformer layers = iterations of gradient descent on $\mathcal{E} + \mathcal{F}$. Attn = gradient of interaction energy; MLP = gradient of potential energy.

2. YuriiFormer Architecture

Replace GD template with **Nesterov acceleration**; preserve Attn/MLP oracles and Lie–Trotter splitting. Two-stream (X_t, V_t) design with negligible overhead.

3. Consistent Empirical Gains

Nesterov+Lie–Trotter **consistently outperforms** nanoGPT on TinyStories and OpenWebText (both small and medium), and improves downstream acc_norm on HellaSwag and ARC-Easy.

Broader message: Optimization-theoretic insights can translate into **principled, practical architectural modifications**.

Conclusion & Discussion Points

Take-aways:

- ① The optimization lens unifies Attn and MLP and gives a **principled vocabulary** for transformer design;
- ② Nesterov acceleration is a **drop-in architectural upgrade** to GPT: same oracles, same splitting, one new stream;
- ③ **Lie–Trotter** $>$ Euler consistently; **Nesterov** $\geq$ Polyak consistently;

Discussion questions:

- Could this perspective inform molecular model design?
- Next steps: scale up; explore second-order (Newton-style) templates

“Transformers are optimizers. Let’s build better ones.”

Zimin, Polyanskiy & Rigollet, arXiv:2601.23236